

```

1  import torch
2  import torch.nn as nn
3  import torch.nn.functional as F
4
5  class MBConv1D(nn.Module):
6      def __init__(self, in_channels, out_channels, expansion=4, stride=1):
7          super().__init__()
8          mid_channels = in_channels * expansion
9          self.use_res_connect = (stride == 1 and in_channels == out_channels)
10
11         self.expand = nn.Sequential(
12             nn.Conv1d(in_channels, mid_channels, kernel_size=1, bias=False),
13             nn.BatchNorm1d(mid_channels),
14             nn.SiLU()
15         )
16         self.depthwise = nn.Sequential(
17             nn.Conv1d(mid_channels, mid_channels, kernel_size=3, stride=stride,
18                 padding=1, groups=mid_channels, bias=False),
19             nn.BatchNorm1d(mid_channels),
20             nn.SiLU()
21         )
22         self.se = nn.Sequential(
23             nn.AdaptiveAvgPool1d(1),
24             nn.Conv1d(mid_channels, mid_channels // 4, kernel_size=1),
25             nn.SiLU(),
26             nn.Conv1d(mid_channels // 4, mid_channels, kernel_size=1),
27             nn.Sigmoid()
28         )
29         self.project = nn.Sequential(
30             nn.Conv1d(mid_channels, out_channels, kernel_size=1, bias=False),
31             nn.BatchNorm1d(out_channels)
32         )
33
34         def forward(self, x):
35             out = self.expand(x)
36             out = self.depthwise(out)
37             se_weight = self.se(out)
38             out = out * se_weight
39             out = self.project(out)
40             if self.use_res_connect:
41                 return x + out
42             return out
43
44     class EfficientNet1DLSTM(nn.Module):
45         def __init__(self, csi_input_channels, meta_input_size, num_classes):
46             super().__init__()
47             self.stem = nn.Sequential(
48                 nn.Conv1d(csi_input_channels, 32, kernel_size=3, stride=1, padding=1),
49                 nn.BatchNorm1d(32),
50                 nn.SiLU()
51             )
52             self.mbconv1 = MBConv1D(32, 24, stride=2)
53             self.mbconv2 = MBConv1D(24, 40, stride=2)
54             self.mbconv3 = MBConv1D(40, 80, stride=2)
55             self.pool = nn.AdaptiveAvgPool1d(1)
56
57             self.lstm = nn.LSTM(meta_input_size, hidden_size=64, num_layers=2, batch_first=True,
58                 bidirectional=True)
59             self.fc = nn.Linear(80 + 64*2, num_classes)
60
61         def forward(self, csi_seq, meta_seq):
62             # csi_seq shape: (B, W, C), transpose for Conv1d: (B, C, W)
63             x = csi_seq.permute(0, 2, 1)
64             x = self.stem(x)
65             x = self.mbconv1(x)
66             x = self.mbconv2(x)
67             x = self.mbconv3(x)
68             x = self.pool(x).squeeze(-1)
69
70             _, (h_n, _) = self.lstm(meta_seq)
71             h_n = torch.cat((h_n[-2], h_n[-1]), dim=1)

```

```
71     out = torch.cat([x, h_n], dim=1)
72     out = self.fc(out)
73     return out
74
```